The Geometry of Network Healing

Geometric deep learning for business-aware, self-healing telecom networks

Open mathematics, open code, NVIDIA-accelerated

David Kypuros

October 2026 · v0.1

Abstract

A telecom network is not a grid. It is a heterogeneous directed graph whose node labels carry no information, whose faults propagate along typed dependency edges, and whose business value is concentrated in a few revenue-dense cells. This monograph derives, from the geometric deep learning blueprint, the minimal learning system that detects silent degradation without alarms, ranks root causes across RAN, Core and Transport from one service-layer trigger, and orders remediation by revenue at risk under a resource budget. Every numbered equation maps to a module in the accompanying open-source repository, and every claim is measured on a synthetic digital twin with injected faults against graph-free baselines. The point is simple: the model is mathematics anyone can implement; the operator's moat is the topology and the history, not the network.

Contents

1 The claim **3**

2 The domain **5**
2.1 A network as a heterogeneous directed graph 5
2.2 Adjacency and signals 5
2.3 Business value as a node attribute 6

3 Symmetry and the blueprint **7**
3.1 The symmetry group is permutation, not rotation 7
3.2 Message passing is the equivariant operator class 7
3.3 Relational form with transposed relations 8
3.4 Input encoding and read-outs 8

4 Fault propagation as directed diffusion **9**
4.1 Injection 9
4.2 Nominal signal 9
4.3 Observation model 9
4.4 Propagation 10
4.5 Why this is a diffusion and why that matters 10

5 Silent degradation without alarms **11**
5.1 What an alarm is 11
5.2 Two graph-aware scores 11

6 Root cause as an inverse problem **12**
6.1 The forward map and its ill-posedness 12
6.2 Learned inverse 12
6.3 Loss 13
6.4 Counterfactual check 13

7 Business intent as constrained selection **14**
7.1 Separation of concerns 14
7.2 Revenue at risk 14
7.3 Selection under a budget 14

8 The closed loop **16**
8.1 Decomposition 16
8.2 Measuring the loop 17

9 Experiments **18**
9.1 Protocol . . . 18
9.2 Results . . . 19
9.3 Reading the table . . . 19
9.4 What would change these numbers . . . 20

10 The NVIDIA path to scale **21**
10.1 What changes mathematically: sampling . . . 21
10.2 Where the pieces run . . . 21
10.3 What we did and did not verify . . . 21

11 Relation to the industry design **22**
11.1 Stage by stage . . . 22
11.2 What transfers and what does not . . . 23
11.3 One honest difference in the detection story . . . 23

A Equation-to-code map **24**

B Reproducing every number **26**

Chapter 1

The claim

Same fault, different response

Two cells degrade by the same amount. One carries a stadium on match day; the other carries a car park at 3 a.m. Today both enter the same resolution queue because the network has no vocabulary for the difference. Cross-domain triage, deciding whether a service-layer symptom comes from the radio, the transport ring or the core, is still largely a human walking three dashboards.

Recent industry work on autonomous networks has shown that a graph neural network over a temporal digital twin, with a business-intent layer on top and standards-based orchestration below, can close that loop at Level 4 autonomy. The public descriptions of such systems are impressive and also, on inspection, built from three ingredients:

1. a graph database that holds the topology and its history (engineering);
2. a graph neural network that reads the topology and the KPI history (mathematics);
3. an intent vocabulary that lets business priority reorder remediation (a small optimisation).

Items 2 and 3 are not proprietary. They are consequences of a general blueprint, geometric deep learning [1]: identify the symmetry group of your domain, build layers that respect it, pool locally, read out invariantly. This document does that derivation for a telecom network, writes down every operator, implements it in open code, and measures it. Item 1 is real engineering at operator scale and is discussed honestly in Chapter 10, where we describe the NVIDIA path (GPU-resident graph storage with WholeGraph, GPU sampling with cuGraph-PyG) that we assume for scale-out.

What is and is not claimed

- **Claimed.** The learning stack of a business-aware GNN-healing system is reproducible from the equations in Chapters 2–7 by anyone with the topology and the KPI history. Our implementation detects injected silent degradations earlier than static-threshold alarms, ranks the true origin first in the large majority of episodes, and changes the remediation order when revenue changes (Chapter 9).
- **Not claimed.** That the synthetic twin is a real network. Its purpose is to be *falsifiable*: every number in Chapter 9 is produced by one command from a fixed seed, and the generator and fault model are the first thing a reader should change.
- **Not claimed.** Any particular vendor's results. No proprietary data or code is used.

How to read this

Each chapter ends in numbered equations. Appendix A maps every equation to the Python module that implements it. Chapter 9 is generated from the repository's `results/results.md`. If you change an equation, change the module, rerun, and the table changes.

Chapter 2

The domain

2.1 A network as a heterogeneous directed graph

Definition 2.1 (Network graph)**.** *A network is a tuple $G = (\mathcal{V}, \mathcal{E}, \tau, \rho)$ where $\mathcal{V}$ is a finite set of N nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ a set of directed edges, $\tau : \mathcal{V} \to \mathcal{T}$ a node-type map and $\rho : \mathcal{E} \to \mathcal{R}$ an edge-relation map.*

$$G = (\mathcal{V}, \mathcal{E}, \tau, \rho), \qquad \mathcal{T} = \{\mathsf{cell}, \mathsf{gnb}, \mathsf{router}, \mathsf{link}, \mathsf{upf}, \mathsf{amf}, \mathsf{smf}, \mathsf{service}\}. \tag{2.1}$$

Three choices in (2.1) matter and are easy to get wrong.

Edges point in the direction a fault propagates. An edge $v \to u$ means "a degradation at v is felt at u". A fibre link feeds a router; a core router feeds an aggregation router; an aggregation router backhauls a gNB; a gNB serves cells; the AMF controls gNBs; the SMF controls UPFs; a core router feeds a UPF; a UPF terminates services; cells carry services. This is the *causal* orientation, not the traffic orientation, and it is what makes Chapter 4 a diffusion and Chapter 6 an inverse problem.

Links are nodes. A transport link carries its own KPIs (latency, loss, utilisation) and can be the origin of a fault. Modelling it as an edge attribute would make it impossible to name as a root cause. So links are first-class nodes of type link with a single outgoing relation to the router they feed.

Relations are typed. $\mathcal{R}$ has nine members. "Link feeds router" and "AMF controls gNB" must not share a weight matrix: a signalling storm and a fibre cut leave different signatures on their neighbours.

2.2 Adjacency and signals

Let $A \in \{0,1\}^{N \times N}$ be the cause→effect adjacency and A_r its restriction to relation $r \in \mathcal{R}$:

$$A_{vu} = \mathbf{1}[(v,u) \in \mathcal{E}], \qquad A = \sum_{r \in \mathcal{R}} A_r, \qquad \mathcal{N}_r(v) = \{u : (u,v) \in \mathcal{E},\ \rho(u,v) = r\}. \tag{2.2}$$

Each node carries a KPI vector $x_v(t) \in \mathbb{R}^K$ at discrete time t, standardised so that nominal behaviour is near zero. We use $K = 4$ (throughput, latency, loss, utilisation) for every type; the tensors stay rectangular and the *direction* in which degradation shows up is what differs by type (Chapter 4). The full state is a signal on the graph,

$$X(t) \in \mathbb{R}^{N \times K}, \qquad X_{v\cdot}(t) = x_v(t). \tag{2.3}$$

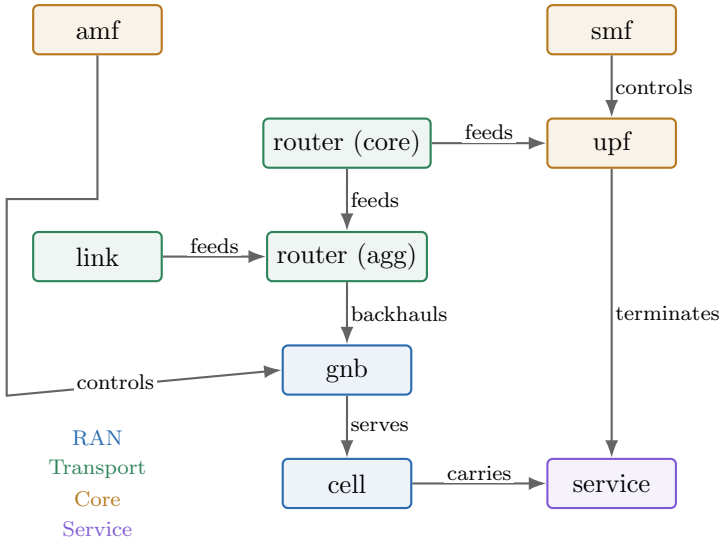


Figure 2.1: The node types and relations of (2.1), colour-coded by domain. Arrows point in the direction a fault propagates. A service is the trigger surface: a service-layer symptom is the only input the healing loop needs.

2.3 Business value as a node attribute

Every cell c carries a revenue density $\text{rev}_c \geq 0$. It is heavy-tailed on purpose (log-normal in the generator): a few cells carry most of the revenue, which is the regime where "same fault, different response" is not a slogan but a measurable reordering (Chapter 7). Revenue is an attribute of the twin, not of the model: the model never sees it, and that separation is the whole design.

Chapter 3

Symmetry and the blueprint

3.1 The symmetry group is permutation, not rotation

The geometric deep learning blueprint [1] asks first: under which transformations of the input must the answer be unchanged (invariant) or transform predictably (equivariant)? For a protein in space the group is $SE(3)$: rotate the molecule and the predicted structure rotates with it. For an image it is translation. For a network graph the only freedom in the representation is *how we numbered the nodes.* Node 17 is a cell because $\tau(17) = \mathsf{cell}$, not because of the integer 17.

Definition 3.1 (Permutation action)**.** *Let* $\mathrm{Sym}(N)$ *act on a network by relabelling. For* $\pi \in \mathrm{Sym}(N)$ *with permutation matrix* P_π*,*

$$\pi \cdot (X, A, \tau) = (P_\pi X,\ P_\pi A P_\pi^\top,\ \tau \circ \pi^{-1}). \tag{3.1}$$

Assumption 3.1 (Equivariance requirement)**.** *A per-node predictor* f *(forecast, degradation, root-cause score) must satisfy*

$$f(\pi \cdot (X, A, \tau)) = P_\pi\, f(X, A, \tau) \qquad \forall\, \pi \in \mathrm{Sym}(N), \tag{3.2}$$

and a graph-level quantity ("is there a fault anywhere") must be invariant, $f(\pi \cdot \cdot) = f(\cdot)$*.*

This is not a nicety. A model that violates (3.2) has learned something about the integer labels, which carry no information, and will fail on the same network exported with a different ordering. The test suite checks (3.2) numerically on the implemented model (`tests/test_model.py`).

Remark 3.1 (Where the gauge story does and does not apply)**.** *Lectures on geometric deep learning spend a long time on* $SO(3)$*, gauge transformations and manifolds because those are the hard cases: there, the symmetry acts on the* feature values *as well as on positions, and one needs irreducible representations and spherical harmonics to build equivariant layers. For a network, the group acts only on the* indexing*. Permutation equivariance is achieved by a structural rule, below, and nothing more exotic is needed. We note this because it is tempting to reach for heavier machinery than the problem demands.*

3.2 Message passing is the equivariant operator class

Proposition 3.1 (Local aggregation is equivariant)**.** *Any layer of the form*

$$h_v^{(\ell+1)} = \phi\Big(h_v^{(\ell)},\ \bigoplus_{u \in \mathcal{N}(v)} \psi\big(h_u^{(\ell)}, h_v^{(\ell)}, e_{uv}\big)\Big), \tag{3.3}$$

with $\bigoplus$ *a permutation-invariant aggregator (sum, mean, max) and* ϕ, ψ *shared across nodes, satisfies* (3.2)*.*

Proof. ϕ and ψ are applied node-wise with shared parameters, so relabelling nodes relabels their outputs. $\mathcal{N}(v)$ is defined by A, which transforms as $P_\pi A P_\pi^\top$, so the neighbourhood of node $\pi(v)$ in the relabelled graph is the relabelled neighbourhood of v; invariance of $\bigoplus$ to the order of its arguments finishes the argument. □

3.3 Relational form with transposed relations

We instantiate (3.3) as a relational graph convolution [2] because the relations in $\mathcal{R}$ carry different physics. Crucially, we also include the *transpose* of every relation. A fault propagates cause→effect, but inferring the cause from observed effects requires information to flow effect→cause (Chapter 6). With $\mathcal{R}' = \mathcal{R} \cup \mathcal{R}^\top$, $|\mathcal{R}'| = 2|\mathcal{R}|$:

$$h_v^{(\ell+1)} = h_v^{(\ell)} + \sigma\left(\mathrm{LN}\left(W_0^{(\ell)} h_v^{(\ell)} + \sum_{r \in \mathcal{R}'} \frac{1}{|\mathcal{N}_r(v)|} \sum_{u \in \mathcal{N}_r(v)} W_r^{(\ell)} h_u^{(\ell)}\right)\right). \tag{3.4}$$

The residual connection and layer norm are optimisation conveniences and do not affect (3.2). Mean aggregation keeps the scale independent of degree, which matters because a core router has many more effect-neighbours than a cell.

3.4 Input encoding and read-outs

The input to the first layer is a window of W past KPI vectors plus a learned type embedding:

$$h_v^{(0)} = \mathrm{MLP}\big(\mathrm{vec}\,[x_v(t-W), \ldots, x_v(t-1)]\big) + E_{\tau(v)}, \qquad E \in \mathbb{R}^{|\mathcal{T}| \times d}. \tag{3.5}$$

After L layers each node holds $h_v^{(L)} \in \mathbb{R}^d$ that summarises its own history and its L-hop causal neighbourhood in both directions. Three linear read-outs, shared across nodes, produce the quantities the rest of the system consumes:

$$\hat{x}_v(t) = W_f h_v^{(L)} \in \mathbb{R}^K, \qquad \hat{d}_v(t) = w_d^\top h_v^{(L)} \in \mathbb{R}, \qquad z_v(t) = w_z^\top h_v^{(L)} \in \mathbb{R}. \tag{3.6}$$

$\hat{x}$ is a one-step forecast (self-supervised), $\hat{d}$ an estimate of the latent degradation field of Chapter 4, and z a root-cause logit used in Chapter 6.

Remark 3.2 (Depth equals causal reach)**.** *L layers see L hops. The longest causal chain in Figure 2.1 is link→router$_{core}$→router$_{agg}$→gNB→cell→service, five hops. We use $L = 3$ with transposes, which lets a service-layer symptom and a transport-layer origin exchange information in one forward pass through the shared router. For larger topologies L should track the diameter of the dependency graph, or sampling should be used (Chapter 10).*

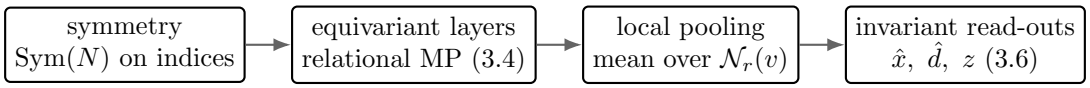


Figure 3.1: The blueprint instantiated. Nothing in the pipeline refers to a node's integer id.

Chapter 4

Fault propagation as directed diffusion

The learning problem is only well posed if we say what a fault *is*. We commit to a simple, explicit generative model. Its purpose is twofold: it produces ground truth for training and evaluation, and it is the physical prior that the network of Chapter 3 is asked to invert. Readers with real incident data should replace this chapter with their data; the rest of the document does not change.

4.1 Injection

A fault is a tuple (v_0, t_0, s, T_r): origin node, onset step, peak severity, ramp length. Its injection into the network is a ramp,

$$s(t) = s \cdot \min\Big(1,\ \frac{t - t_0 + 1}{T_r}\Big)\mathbf{1}[t \geq t_0]\,. \tag{4.1}$$

Long ramps are the *silent* regime: the origin's own KPIs stay inside any static threshold for tens of steps while the pattern across its neighbours is already distinctive.

4.2 Nominal signal

Each KPI follows a diurnal cycle plus coloured noise,

$$n_{vk}(t) = a_{vk}\sin\Big(\frac{2\pi t}{P} + \varphi_{vk}\Big) + \epsilon_{vk}(t), \qquad \epsilon_{vk}(t) = r\,\epsilon_{vk}(t-1) + \eta_{vk}(t), \quad \eta \sim \mathcal{N}(0, \sigma^2). \tag{4.2}$$

The AR(1) term is what makes per-node z-scores unreliable: consecutive excursions are correlated, so a graph-free detector that fires on a short run of outliers has a high false-alarm rate (Chapter 9).

4.3 Observation model

Degradation is a scalar latent field $d(t) \in \mathbb{R}^N_{\geq 0}$. It enters the observed KPIs through a type-specific effect direction $e_\tau \in \mathbb{R}^K$: a congested link shows latency and loss, a sleeping cell shows throughput collapse, a signalling storm shows control-plane utilisation.

$$x_v(t) = n_v(t) + d_v(t)\,e_{\tau(v)}. \tag{4.3}$$

4.4 Propagation

The field evolves as a damped, directed heat flow on the cause→effect graph:

$$d(t+1) = \rho\, d(t) + \beta\, D_{\text{in}}^{-1} A^{\top} d(t) + s(t)\, \mathbf{1}_{v_0}, \qquad (D_{\text{in}})_{uu} = \max(1, \textstyle\sum_v A_{vu}). \tag{4.4}$$

ρ is memory (a degraded router does not recover in one step), β is coupling (how much of a cause's degradation its effects inherit per step), and the in-degree normalisation says an effect with many causes averages them rather than summing them.

Proposition 4.1 (Steady state and attenuation)**.** *With constant injection s at an origin with no causes of its own, the origin reaches $d^{\star}_{v_0} = s/(1-\rho)$, and a node k hops downstream along a single chain reaches $d^{\star}_{k} = \left(\frac{\beta}{1-\rho}\right)^{k} d^{\star}_{v_0}$. Faults attenuate per hop iff*

$$\frac{\beta}{1-\rho} < 1. \tag{4.5}$$

Proof. At steady state $d = \rho d + \beta P d + s\mathbf{1}_{v_0}$ with $P = D_{\text{in}}^{-1} A^{\top}$. For the origin, $(Pd)_{v_0} = 0$, giving the first claim. For a chain, $(Pd)_k = d_{k-1}$, so $d_k(1-\rho) = \beta d_{k-1}$. □

We set $\rho = 0.85$, $\beta = 0.10$, so each hop keeps two thirds of its cause's degradation and the blast radius of a core fault reaches the service layer with a recognisable but diminished signature. The repository asserts (4.5) in its tests; violating it produces an exploding, unphysical twin.

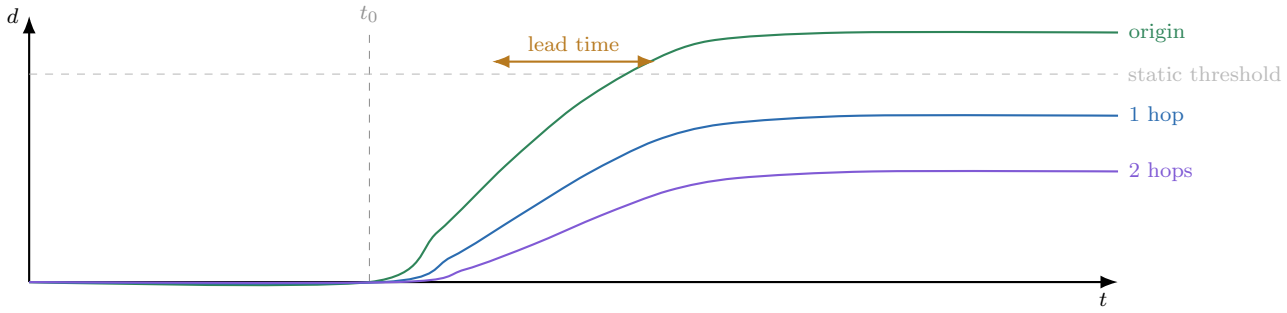


Figure 4.1: The silent regime of (4.1)–(4.4). The origin crosses a static threshold late; the joint pattern across hops is distinctive early. That gap is what the graph buys.

4.5 Why this is a diffusion and why that matters

Equation (4.4) is a discrete heat equation with a directed, degree-normalised Laplacian $I - P$ and a source term. The forward map from injection history to the observed field is linear and known. The learning problem is therefore not "find a pattern" but "invert a known operator from noisy, indirectly observed data" (Chapter 6). That framing tells us what the network must contain: the transposed relations, since $P^{\top}$ is the adjoint of the propagation operator, and enough depth to span the support of the operator's impulse response.

Chapter 5

Silent degradation without alarms

5.1 What an alarm is

A static alarm fires when some KPI of some node leaves a fixed band for a short run:

$$t_{\text{alarm}} = \min\Big\{t : \ \exists v, \ \max_{k} |x_{vk}(t')| > \theta_{\text{static}} \ \ \forall t' \in [t-m+1, t]\Big\}. \tag{5.1}$$

It is graph-free and history-free. Under (4.3) it fires when $d_v e_{\tau(v),k}$ plus noise exceeds θ_{static} at a single node, which in the silent regime is late. A per-node rolling z-score replaces the fixed band with a rolling one; it is earlier but, under the AR(1) noise of (4.2), it is also wrong often. Both are baselines in Chapter 9.

5.2 Two graph-aware scores

The network of Chapter 3 gives two per-node quantities. The forecast residual is label-free:

$$r_v(t) = x_v(t) - \hat{x}_v(t), \qquad \hat{x}_v(t) \text{ from (3.6) on the window } [t-W, t-1]. \tag{5.2}$$

It is the right score when no labelled history exists. Its weakness is that a one-step forecaster on a slow ramp absorbs the drift: the residual is the per-step increment, not the accumulated degradation. The degradation estimate uses the twin's latent field as supervision and reads the accumulated state directly:

$$a(t) = \max_{v \in \mathcal{V}} \hat{d}_v(t), \qquad t_{\text{gnn}} = \min\{t : \ a(t') > \theta \ \ \forall t' \in [t-m+1, t]\}. \tag{5.3}$$

The threshold θ is not a KPI threshold. It is calibrated once on nominal episodes as a margin above the largest $\hat{d}$ the model ever produced on healthy data, so false alarms on healthy calibration data are zero by construction. In production the calibration set is the operator's healthy history; the supervision for $\hat{d}$ comes from labelled incidents or from a simulator of the operator's own topology.

Remark 5.1 (No alarm is required)**.** *Nothing in* (5.3) *refers to a per-node band. The detector looks at the whole field $\hat{d}(t)$, which the network computes from the KPI windows of every node and their causal neighbourhoods. A service-layer symptom with no alarm anywhere is a valid trigger.*

Chapter 6

Root cause as an inverse problem

6.1 The forward map and its ill-posedness

Unrolling (4.4) from onset, the degradation field is a linear function of the injection history,

$$d(t) = \sum_{k=0}^{t-t_0} M^k \, \mathbf{1}_{v_0} \, s(t-k), \qquad M = \rho I + \beta P, \quad P = D_{\text{in}}^{-1} A^\top. \tag{6.1}$$

The root-cause question is: given noisy observations of X (not d), recover v_0. This is an inverse problem on the graph and it is ill-posed in the usual ways. Two origins with the same downstream reach are indistinguishable from their effects alone (a link and the router it feeds differ only in the link's own KPIs). The observation (4.3) mixes d with diurnal and coloured noise. And M^k smooths: the impulse response spreads over k hops, so late observations carry less positional information than early ones.

What the structure of (6.1) tells us is which inductive biases a learned inverse must contain. The adjoint of the forward operator is $M^\top = \rho I + \beta P^\top$, which propagates information effect→cause along the transposed relations. That is why $\mathcal{R}^\top$ is in (3.4): a network without transposes can only smooth downstream and cannot localise.

6.2 Learned inverse

We do not invert (6.1) analytically; real networks are not exactly linear, the observation noise is not exactly Gaussian, and the ramp is unknown. We learn a conditional distribution over origins from the relational embedding. With the logit $z_v(t)$ from (3.6),

$$p(v_0 = v \mid X_{[t-W,t)}, G) = \frac{\exp z_v(t)}{\sum_{u \in \mathcal{V}} \exp z_u(t)}, \tag{6.2}$$

and the root-cause ranking at time t is $\mathcal{V}$ sorted by $z_v(t)$ descending. The softmax is over *all nodes of all types*: the network is asked "which single node" across RAN, Core and Transport, which is the cross-domain triage question.

Remark 6.1 (Node scoring is link prediction with one anomaly node)**.** *A common production formulation adds an* anomaly *node a to the property graph, connected to every symptomatic entity, and trains a link predictor to score the edge $a \to v$ labelled "originates from". With a bilinear scorer $z_v = h_a^\top M h_v$ and a single anomaly node per incident, h_a is a constant for the episode and z_v is a linear read-out of h_v: exactly* (3.6)*. The two formulations differ only when several concurrent incidents must be attributed separately, in which case the anomaly-node version is the natural generalisation.*

Remark 6.2 (Equivariance makes the ranking well defined)**.** *Because z is a per-node output of an equivariant map* (3.2)*, relabelling the network permutes the ranking consistently. The claim "node 17 is the root cause" is really the claim "the node with this history and this neighbourhood is the root cause", which is what an operator means.*

6.3 Loss

The three heads are trained jointly on windows drawn from simulated episodes,

$$\mathcal{L} = \underbrace{\frac{1}{NK}\left\|\hat{X}(t) - X(t)\right\|_F^2}_{\text{forecast}} + \underbrace{\frac{1}{N}\left\|\hat{d}(t) - d(t)\right\|_2^2}_{\text{degradation}} + \lambda \underbrace{\mathbf{1}[t \geq t_0 + \delta]\left(-\log p(v_0 \mid \cdot)\right)}_{\text{root cause}}, \tag{6.3}$$

with the root-cause term switched on δ steps after onset, when some signal has accumulated, and switched off entirely on nominal episodes. The forecast term is included even though detection uses $\hat{d}$: it is label-free, it regularises the embedding toward the KPI dynamics, and it is the term that survives when a deployment has no incident labels.

6.4 Counterfactual check

Because the twin is explicit, a ranking can be verified rather than trusted: remove the injection at the top-ranked node and re-simulate. If the field decays, the ranking was right. The closed loop of Chapter 8 does exactly this, and the exposure numbers in Chapter 9 are computed from the re-simulation, not from the model's own confidence.

Chapter 7

Business intent as constrained selection

7.1 Separation of concerns

The model of Chapters 3–6 never sees revenue. It outputs a ranking of candidate origins and an estimated degradation field. Everything about *which* remediation runs first when recovery resources are constrained lives in a separate layer with its own vocabulary. This is the decoupling that a business-intent extension to an intent ontology provides: the operator declares priority in business terms; the network implementation does not change.

7.2 Revenue at risk

Let $\mathrm{rev}_u \geq 0$ be the revenue density attached to node u (nonzero on cells, zero elsewhere in our twin; a real deployment attaches it wherever revenue is measured), let $w_u \geq 0$ be an operator-declared profile weight from a small catalogue (mass-event venue, transit hub, enterprise-critical, residential), and let $\mathrm{reach}(v)$ be the forward closure of v along cause→effect edges, the blast radius.

$$\mathrm{rev},\, w : \mathcal{V} \to \mathbb{R}_{\geq 0}, \qquad \mathrm{reach}(v) = \{u : \; v \rightsquigarrow u\} \cup \{v\}. \tag{7.1}$$

The revenue at risk of a candidate origin v is the weighted revenue in its blast radius, scaled by the estimated degradation there:

$$R(v) = \sum_{u \in \mathrm{reach}(v)} w_u \,\mathrm{rev}_u \,\max\big(0, \hat{d}_u(t)\big). \tag{7.2}$$

With $w \equiv 1$ this is pure revenue density. With a catalogue of weights and a single-node blast radius it reduces to the "profile weight × revenue density × service impact" utility used in industry intent extensions, where $\hat{d}_u$ plays the role of service impact. Two candidate origins with identical network signatures but different downstream revenue get different R. That is the entire mechanism behind "same fault, different response".

7.3 Selection under a budget

Each candidate v induces a remediation branch with a resource cost $c(v)$ determined by its type (resetting a cell is cheap, scaling a core function is not). Given the top-k root-cause candidates and a budget B of recovery resources, the executed order is the greedy solution of

$$\max_{S \subseteq \text{top-}k} \sum_{v \in S} R(v) \quad \text{s.t.} \quad \sum_{v \in S} c(v) \leq B, \qquad \text{executed in decreasing order of } R(v)/c(v), \tag{7.3}$$

with at most one high-risk change per domain per healing cycle (a change-risk concurrency policy); the remaining branches are held as ranked fallbacks for the verify stage. This is a knapsack with a ratio-greedy heuristic, deliberately simple so that an operator can read the rule in one line and change it. The output is serialised as an intent object whose targets carry domain, action, priority, revenue at risk and resource units, in the shape of a TMF921 intent with a business-priority extension.

Remark 7.1 (The concurrency policy is per-process here)**.** *The one-change-per-domain rule in* (7.3) *is enforced inside a single selection call, which is correct for the single-loop experiments of Chapter 9 and insufficient in deployment: several sites healing concurrently each run their own selection and none of them sees the others. The policy then needs a distributed lock per domain with an expiry, backed by an authoritative concurrency gate in the orchestrator, since an advisory lock alone can admit exactly the two simultaneous changes the rule forbids. This gap and the state it implies are recorded in* `docs/state-and-coordination.md`*.*

Remark 7.2 (What the business layer does not do)**.** *It does not move the root-cause ranking. If the model is wrong about the origin, revenue weighting makes the wrong remediation run first with more conviction. The guard against that is the counterfactual check of Chapter 6 and the recovery measurement of Chapter 8, not the intent layer.*

Chapter 8

The closed loop

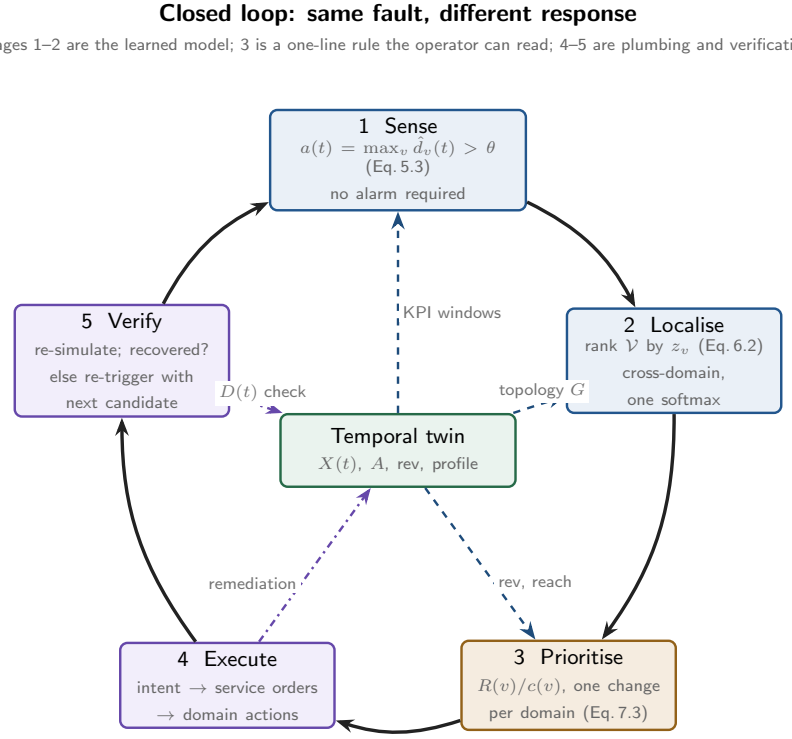


Figure 8.1: Five stages around a twin. The only input is the KPI stream; the only output is an ordered set of per-domain actions. Stages 1–2 are the learned model, stage 3 a one-line rule, stages 4–5 plumbing and verification.

8.1 Decomposition

An intent with n ordered targets decomposes into n service orders, one per target, each carrying the domain (RAN, Core, Transport), the target node and the remediation operation. In the repository the intent is TMF921-shaped and the orders are TMF641-shaped: the field names follow the standards so that a real orchestrator could be substituted for the mock, but we do not claim conformance to either specification. The mock orchestrator records each order and returns the step at which the highest-priority action takes effect.

8.2 Measuring the loop

The twin makes the loop measurable. We re-simulate the same fault with the injection removed from the effective remediation step onward and record the recovery time, and we integrate the revenue-weighted degradation over the episode:

$$\text{Loss} = \sum_{t} \sum_{u \in \mathcal{V}} \text{rev}_u \, d_u(t). \tag{8.1}$$

Comparing (8.1) for a GNN-timed remediation against a threshold-timed one turns the lead time of Chapter 5 into a business number on the same twin. If the model's top-ranked origin is wrong, the remediation is applied to the wrong node, the injection is not removed, and the GNN-timed loss is *not* reduced; the metric therefore penalises wrong root causes rather than rewarding confident ones.

Chapter 9

Experiments

Every number below is produced by `make pipeline` from a fixed seed and rendered into this chapter by `scripts/results_to_tex.py`. Change an equation, change the module, rerun, and the table changes.

9.1 Protocol

A single synthetic twin (Chapter 2) is generated. Episodes of T steps are simulated; a fraction are nominal, the rest carry one fault drawn from the catalogue of Chapter 4 with a random origin of the matching type, a random onset in $[T/4, T/2)$, and a random severity and ramp. Episodes are split 75/25 into train and test, and a separate set of nominal-only episodes with its own seed plays the role of clean operating history. The model of Chapter 3 is trained with the loss (6.3). Every detector, the GNN field detector (5.3), the service-layer z-score trigger, and the per-node rolling z-score, is calibrated on the *same* nominal episodes with the same margin, so each has zero false alarms on calibration data by construction. Because that calibration is strict (a maximum over every node and step of clean history), both z-score detectors are also reported at the textbook $z = 3$ with a three-step confirmation gate, which is the setting an operator would more likely deploy, together with its false-alarm cost. The static threshold is fixed at 2.5 standardised units.

Metrics (Chapter 8 and below): a detector's delay is the first firing at or after onset; a firing before onset is counted as a false alarm. Lead time is reported only over episodes where the baseline fired, and the detection-rate gap is reported separately.

$$\text{lead} = t_{\text{baseline}} - t_{\text{gnn}} \tag{9.1}$$

$$\text{hit@}k = \mathbf{1}[v_0 \in \text{top-}k \text{ of the ranking at } t_{\text{gnn}} + \delta] \tag{9.2}$$

$$\text{triage@}k = 1 - \frac{k}{|\{u : \; d_u(t_{\text{alarm}} + \delta) > 0.5\}|} \tag{9.3}$$

The triage denominator is the set of nodes that are actually degraded when a static alarm would have fired: the set a war room would face. Exposure is (8.1) integrated over the episode, with the remediation applied at the GNN-timed step through the verify-and-re-trigger loop, or at the threshold-timed step.

9.2 Results

Metric	Value
Fault episodes / nominal episodes	32 / 8
Detection rate: GNN / static threshold	1.00 / 1.00
Detection rate: service z (calibrated $z = 4.1$ / textbook $z = 3$)	0.00 / 0.31
Detection rate: per-node z (calibrated $z = 8.0$ / textbook $z = 3$)	0.00 / 1.00
False alarms on nominal: GNN / thr. / service z cal. / service z=3 / node z cal. / node z=3	0 / 0 / 0 / 4 / 0 / 8
Pre-onset false alarms on fault episodes, same order	0 / 0 / 0 / 3 / 0 / 30
Detection delay after onset (steps): GNN / static threshold	22.9 / 32.6
Detection delay (steps): service z cal. / z=3; node z cal. / z=3	– / 56.2; – / 9.6
Lead time of GNN vs static threshold (steps, where it fired)	9.8
Lead time of GNN vs service z (calibrated / z=3)	– / 32.0
Lead time of GNN vs per-node z (calibrated / z=3)	– / -13.3
Root cause hit@1 / hit@3	1.00 / 1.00
Healed within 3 verify-and-re-trigger cycles / mean cycles	1.00 / 1.00
War-room set at alarm time (nodes) / triage reduction@3	13.9 / 0.35
Revenue-weighted exposure: GNN-timed / threshold-timed healing	328.3 / 573.1

Fault type	n	delay GNN	delay thr.	delay service z=3	hit@1	hit@3
amf signaling storm	7	27.9	45.0	46.5	1.00	1.00
fiber degrade	7	22.7	28.6	112.0	1.00	1.00
router congestion	7	22.3	32.3	6.0	1.00	1.00
sleeping cell	8	18.5	25.4	100.0	1.00	1.00
upf overload	3	24.7	33.3	65.0	1.00	1.00

Twin: 74 nodes, 130 edges. Detection threshold $\theta = 1.139$. Config: episodes=160, epochs=30, T=160, window=8, seed=0. Runtime 647.1 s on CPU.

9.3 Reading the table

- **Lead time.** The quantity the industry calls silent-degradation lead time is the GNN-versus-static-threshold row. Positive is earlier. The z-score rows are the honest comparison with cheap statistical detectors: at the calibrated false-alarm budget and at the textbook setting. A service-layer trigger sees a cell's degradation diluted by the number of carriers feeding each service, which is exactly why silent degradation is silent at the service layer; the $z = 3$ variant fires earlier at the cost of pre-onset false alarms, and the table shows both numbers.
- **Root cause.** hit@1 is the fraction of episodes where the single top-ranked node is the injected origin, across all three domains in one softmax.
- **Healing cycles.** Because a remediation only removes the injection at the true origin, a wrong top-1 costs a cycle. The healed-within-3 rate is the end-to-end success of the loop, not of the model alone.
- **Exposure.** Revenue-weighted degradation integrated over the episode. The ratio of the two exposure numbers is the business version of the lead time.

9.4 What would change these numbers

The twin is small and the fault model is simple; both are the first things to vary. Longer ramps widen the GNN's advantage; stronger coupling β narrows it because symptoms reach the service layer sooner. More node types and more relations make hit@1 harder. Real KPI history replaces (4.2) and will be less forgiving than AR(1) noise. None of these change the equations.

Chapter 10

The NVIDIA path to scale

Everything above runs full-batch on a 74-node twin on a laptop CPU in under two minutes. An operator's twin has 10^5–10^7 nodes and the same number of KPI windows per step. The mathematics does not change; the execution does. We assume NVIDIA hardware and the RAPIDS ecosystem for the scaled deployment and document the path here. The repository's `docs/nvidia.md` carries the installation and versioning detail.

10.1 What changes mathematically: sampling

Full-batch evaluation of (3.4) touches every edge every step. At scale one evaluates the layer on a sampled neighbourhood [3]: for each seed node, draw f_ℓ neighbours per relation at hop ℓ and replace the mean over $\mathcal{N}_r(v)$ with the mean over the sample. The sampled aggregation is an unbiased estimate of the full one, and (3.2) still holds in distribution because the sampler is itself index-blind. What one loses is the exact L-hop receptive field; what one gains is a cost that depends on the fan-out $f_1 f_2 \cdots f_L$ and the batch, not on N.

10.2 Where the pieces run

- **Graph structure and features in GPU memory.** WholeGraph holds the edge list and the KPI window tensor across GPUs (and across nodes of a DGX system over NVLink/NVSwitch), so sampling and gathering never cross to host memory. This is the twin's *in-memory* form; a durable graph store still owns history and versioning.
- **Sampling and gathering.** cuGraph-PyG implements PyTorch Geometric's `GraphStore`/`FeatureStore` and a GPU `NeighborLoader`. The same `HealingGNN` class consumes its mini-batches; `gnn/scale.py` is the adapter.
- **Data preparation.** cuDF replaces pandas for building the node and edge tables and the rolling KPI windows at scale.
- **Training.** PyTorch on CUDA; multi-GPU via distributed data parallel with the sampler sharded by seed nodes. Relational convolution kernels are the stock PyG ones; NVIDIA's accelerated message-passing kernels are picked up when installed.

10.3 What we did and did not verify

The CPU reference path is exercised by the test suite and the pipeline. The cuGraph-PyG and WholeGraph path is *documented and import-guarded*, not exercised in this repository's CI, because the public release is built and tested on a machine without an NVIDIA GPU. A reader with a CUDA 12 Linux host can install the pinned RAPIDS release and run the same pipeline with the adapter; we state this plainly rather than imply a scale result we have not measured.

Chapter 11

Relation to the industry design

Public material from a 2026 TM Forum Moonshot Catalyst on business-aware GNN healing (C26.0.965) describes a production-grade system of the same shape as this repository. We built ours independently from the blueprint, then compared. The comparison is useful in both directions: it shows which parts of the industry system are mathematics that anyone can reproduce, and it shows what we deliberately left out.

11.1 Stage by stage

Stage	Industry design	This repository
Trigger	Service-layer z-score against a clean multi-week baseline with a confirmation gate; the GNN does not detect.	Same service-layer trigger implemented as a baseline (`service_zscore_trigger`); the GNN degradation field (5.3) is offered as a graph-aware alternative and measured against it.
Localise	Link prediction: anomaly nodes live in the property graph and the model scores "originates-from" edges, traversed in reverse from symptoms to origin.	Node scoring with a softmax over all nodes (6.2). Equivalent for a single incident (Remark 6.1); transposed relations in (3.4) are the reverse traversal.
Prioritise	Utility = profile weight $\times$ revenue-density index $\times$ service impact, banded into a priority; a change-risk concurrency policy executes one major change at a time.	$R(v) = \sum_u w_u \,\mathrm{rev}_u \,\hat{d}_u$ over the blast radius, divided by resource cost, under a budget with one change per domain (7.3). Their utility is the single-node special case.
Execute	TMF921 v5 healing intent decomposed into TMF641 v5 service orders, executed by a vendor orchestration engine.	TMF921- and TMF641-shaped objects (field names, not conformance) executed by a mock orchestrator against the simulator.
Verify	A reflection stage re-runs inference, reports complies/degrades, re-triggers without human escalation.	Re-simulation with the remediation applied; if the field does not recover, re-trigger with the next ranked candidate (8.1).
Training data	A 13-stage schema-faithful generator: scripted temporal arcs, domain-specific signal fingerprints, topology-driven blast radii, root-cause vs symptom labels.	A diffusion model on the causal graph (4.4) with type-specific effect vectors (4.3); origin and latent field as labels. Less schema fidelity, more physics.
Scale substrate	A managed temporal graph database and a managed GNN training service.	NVIDIA path: WholeGraph, cuGraph-PyG, cuDF (Chapter 10); durable store left to the reader.

11.2 What transfers and what does not

Everything in the Trigger, Localise, Prioritise and Verify rows is mathematics and is in this repository with tests. The Execute row is plumbing that any orchestrator can provide. The two things the industry system has that this repository does not are a production-identical schema for its synthetic corpus, which makes fine-tuning on real incidents a no-translation step, and the standards contributions themselves (an intent-ontology extension, a new assessment questionnaire, a proposed lead-time indicator). Neither affects the claim of Chapter 1; both matter for deployment.

11.3 One honest difference in the detection story

The industry trigger is a service-layer statistic. That is a sensible production choice: it is cheap, explainable, and needs no labels. Our results (Chapter 9) test whether a graph-aware detector fires earlier on the same episodes with the same calibrated false-alarm budget, because it reads the *pattern across neighbours* rather than a single node's deviation. That is an extension of the industry design, not a replication of it, and we report it as such, including the cases where it does not win.

Appendix A

Equation-to-code map

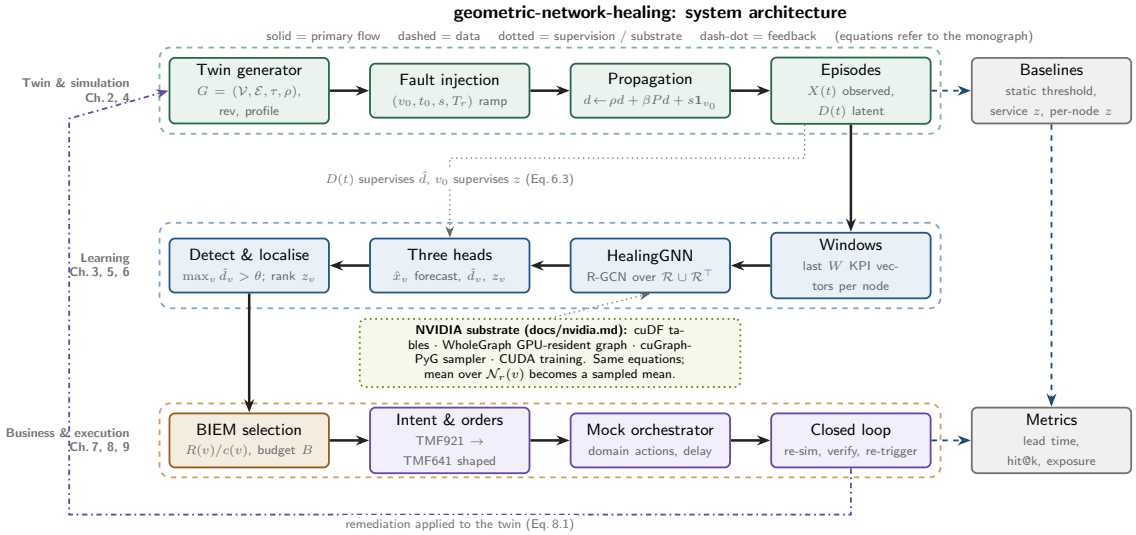


Figure A.1: System architecture of the repository. Equation numbers refer to this document.

Equation	Module	Symbol
(2.1), (2.2), (2.3)	twin/schema.py, twin/generator.py	NodeType, Relation, Twin.adjacency(), Twin.to_pyg()
(7.1)	twin/generator.py	nodes["revenue"], Twin.downstream()
(3.1), (3.2)	tests/test_model.py	numerical equivariance check
(3.3), (3.4), (3.5), (3.6)	gnn/model.py	HealingGNN
(4.1)	sim/faults.py	Fault.injection()
(4.2), (4.3), (4.4)	sim/dynamics.py	simulate()
(4.5)	tests/test_twin_and_sim.py	test_faults_attenuate_per_hop
(5.1)	baselines/alarms.py	threshold_alarm_detection(), zscore_detection()
(5.2), (5.3)	gnn/detect.py	calibrate_threshold(), detect()
(6.1)	sim/dynamics.py	implicit in the recursion
(6.2)	gnn/detect.py	rank_root_causes()
(6.3)	gnn/train.py	train()
(7.2), (7.3)	intent/biem.py	propose_branches(), select_order(), to_tmf921_intent()
(8.1)	orchestration/tmf641.py	closed_loop()
(9.1)–(9.3)	evaluation/metrics.py	lead_time(), hit_at_k(), triage_reduction()
sampling (Ch. 10)	gnn/scale.py	make_cugraph_loader()

Appendix B

Reproducing every number

```
git clone https://github.com/dkypuros/geometric-network-healing
cd geometric-network-healing
make setup          # uv venv, CPU PyTorch + PyG
make test           # 6 tests incl. the equivariance check
make pipeline       # twin -> simulate -> baselines -> GNN -> intent -> closed loop
cat results/results.md
make pdf            # this document
```

The pipeline is seeded; `--seed` changes the twin, the episodes and the initialisation together. `--episodes`, `--epochs`, `--T`, `--window`, `--budget` are the knobs most worth turning. On a CUDA host, `make setup-nvidia` adds the RAPIDS stack and `gnn-healing backend` reports what was detected.

Bibliography

[1] M. M. Bronstein, J. Bruna, T. Cohen, P. Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. arXiv:2104.13478, 2021.

[2] M. Schlichtkrull et al. *Modeling Relational Data with Graph Convolutional Networks*. ESWC 2018.

[3] W. Hamilton, R. Ying, J. Leskovec. *Inductive Representation Learning on Large Graphs*. NeurIPS 2017.

[4] K. Xu, W. Hu, J. Leskovec, S. Jegelka. *How Powerful are Graph Neural Networks?* ICLR 2019.

[5] M. Fey, J. E. Lenssen. *Fast Graph Representation Learning with PyTorch Geometric*. ICLR-W 2019.

[6] TM Forum. *TMF921 Intent Management API* and *TMF641 Service Ordering API*; *Intent Ontology (TIO)*.

[7] NVIDIA. *RAPIDS cuGraph, cuGraph-PyG, WholeGraph* documentation.